NVIDIA Isaac Sim/Lab

Simulace Unitree robotů

Ing. Karel Růžička

15. října 2026

Obsah prezentace

01 NVIDIA Ekosystém

02 Simulace Unitree

03 Instalace simulátoru

04 Použití simulátoru

01 NVIDIA Ekosystém

Proč NVIDIA v robotice?

Limity tradičních simulátorů (Gazebo, PyBullet, MuJoCo) :

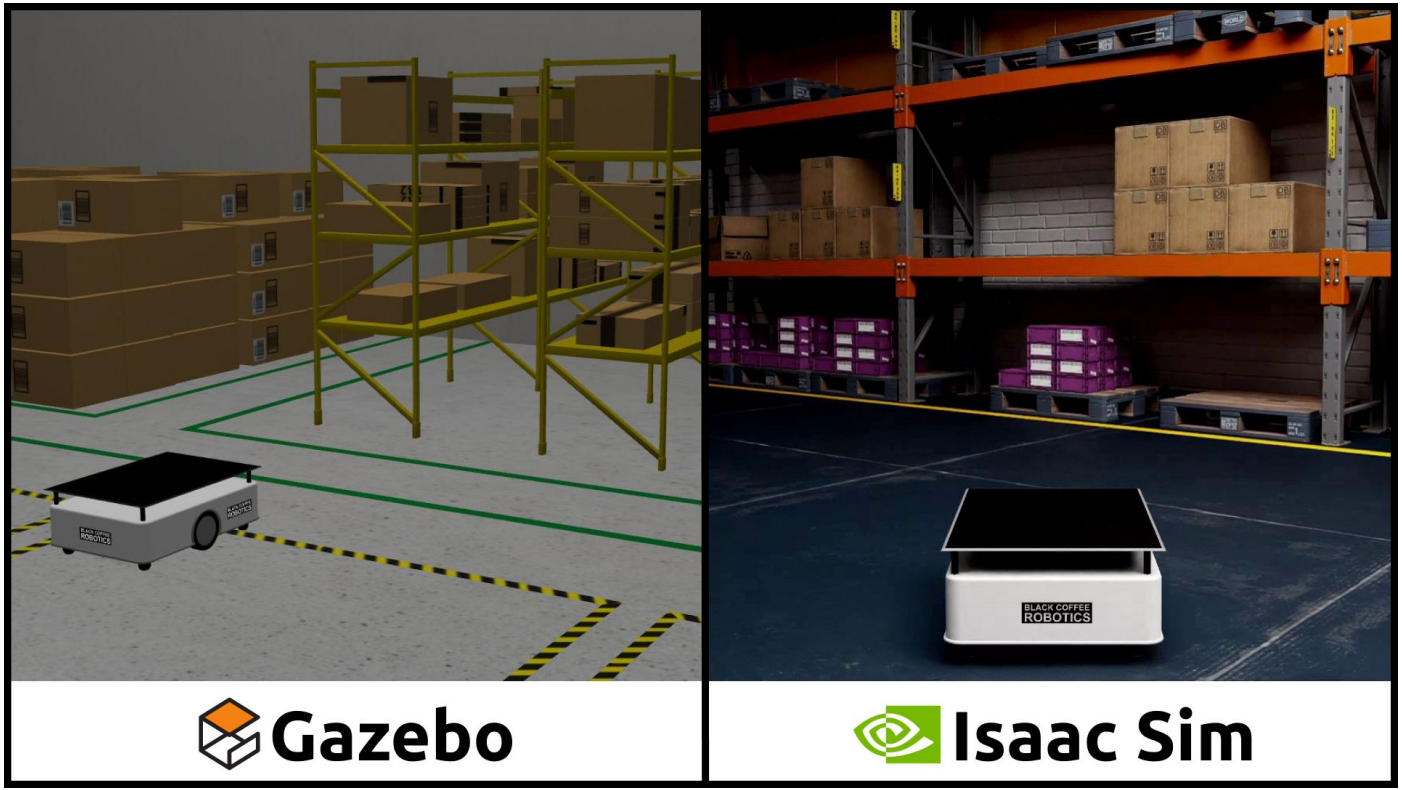
01 **Omezená propustnost CPU architektury**
Trénink RL vyžaduje miliardy kroků: naráží na overhead CPU paralelizace a zpoždění přenosů přes sběrnici PCIe.

02 **Senzorická bariéra**
Nemožnost generovat fotorealistická data (kamery, LiDAR) v reálném čase.

Řešení: End-to-end GPU pipeline
Fyzika, ray-tracing i inference sítě sdílí stejný čip. Eliminace rozdílu mezi simulací a realitou.

Zdroj: blackcoffeerobotics.com

Fotorealičnost simulátorů



Architektonické pilíře NVIDIA Omniverse

OpenUSD

• Standardizovaný formát a ekosystém od Pixaru pro definici 3D scén.

• Nedestruktivní vrstvení a kompozice dat (geometrie, fyzika a materiály v jednotné struktuře).

NVIDIA RTX Renderer

• Fotorealistický rendering v reálném čase (ray tracing a path tracing).

• Fyzikálně přesná simulace světla a odrazů pro virtuální senzory (kamery, LiDAR, radar).

NVIDIA PhysX 5

• Fyzikální engine akcelerovaný přes CUDA (výpočty běží přímo na GPU).

• Masivně paralelizovaná simulace (artikulovaná těla robotů, kolize, deformace).

NVIDIA Isaac Sim vs. Isaac Lab

NVIDIA Isaac Sim

Role: Simulátor, fyzika, rendering

- Vytváření digitálních dvojčat a generování syntetických dat.
- Fyzika PhysX 5 a fotorealistický RTX rendering (LiDAR, kamery).
- Integrace s ROS/ROS 2 a bohaté GUI.

NVIDIA Isaac Lab

Role: Nadstavba nad Sim pro trénink politik (RL, IL)

- Masivní GPU paralelizace (tisíce robotů současně).
- Zaměřeno na lokomoci (humanoidi, čtyřnožci) a manipulaci.
- Odměny (rewards), pozorování, ukončení epizod, randomizace domény.

Klíčové rozdíly

Role
Isaac Sim (Simulátor/Engine) vs. Isaac Lab (Trénovací framework)

Cíl
Testování & data (Sim) vs. Trénink modelů (Lab)

Uživatelské prostředí
GUI/Kód (Sim) vs. Kód-Headless (Lab)

Škálování
Jednotky robotů v detailu (Sim) vs. Tisíce robotů pro učení (Lab)

Historie

01

Isaac Gym

- Samostatný průkopnický projekt firmy NVIDIA
- První důkaz, že end-to-end trénink RL přímo na GPU (bez přenosu na CPU) radikálně urychluje učení robotů
- ***Omezení:*** *Nebyl plně integrován do Omniverse (horší vizualizace a chybějící podpora komplexních senzorů).*

02

Projekt Orbit

- Snaha spojit rychlost Isaac Gym s fotorealistickým prostředím Isaac Simu
- Otevřený (open-source) modulární framework vytvořený komunitou a laboratoří RSL na ETH Zürich

03

Zrození Isaac Lab

- NVIDIA převzala projekt Orbit pod svá křídla jako oficiální standard
- **Orbit byl přejmenován na Isaac Lab** a plně integrován do Omniverse
- Isaac Lab je nyní oficiálním nástupcem Isaac Gym i Orbitu

02 Simulace Unitree

unitree_sim_isaaclab

Nadstavba nad NVIDIA Isaac Lab a NVIDIA Isaac Sim

Hlavní role

- **Verifikační prostředí a virtuální dvojče hardwaru.**
- Není primárně určen k RL tréninku.

Co poskytuje "out-of-the-box"

- **Předpřipravené USD modely:** Přesná kinematika, limity kloubů, aproximace chování motorů Unitree.
- **Komunikační rozhraní (DDS / Unitree SDK):** Řízení přes stejné příkazy (low-level/high-level) jako fyzický hardware.

Výsledek

Software-in-the-Loop (SIL) ověřování

Stejný kód řídí simulaci i reálný hardware bez nutnosti úprav řídicí logiky.

(Model Validation, Data Collection, Playback)

Vztah nadstavby a simulátoru

Aplikace & Logika

unitree_sim_isaaclab

- Hotové 3D/USD modely robotů
- Řízení robota přes DDS

↓

Robotický Framework

NVIDIA Isaac Lab

- Standardní rozhraní pro ovládání simulace (Gym API).

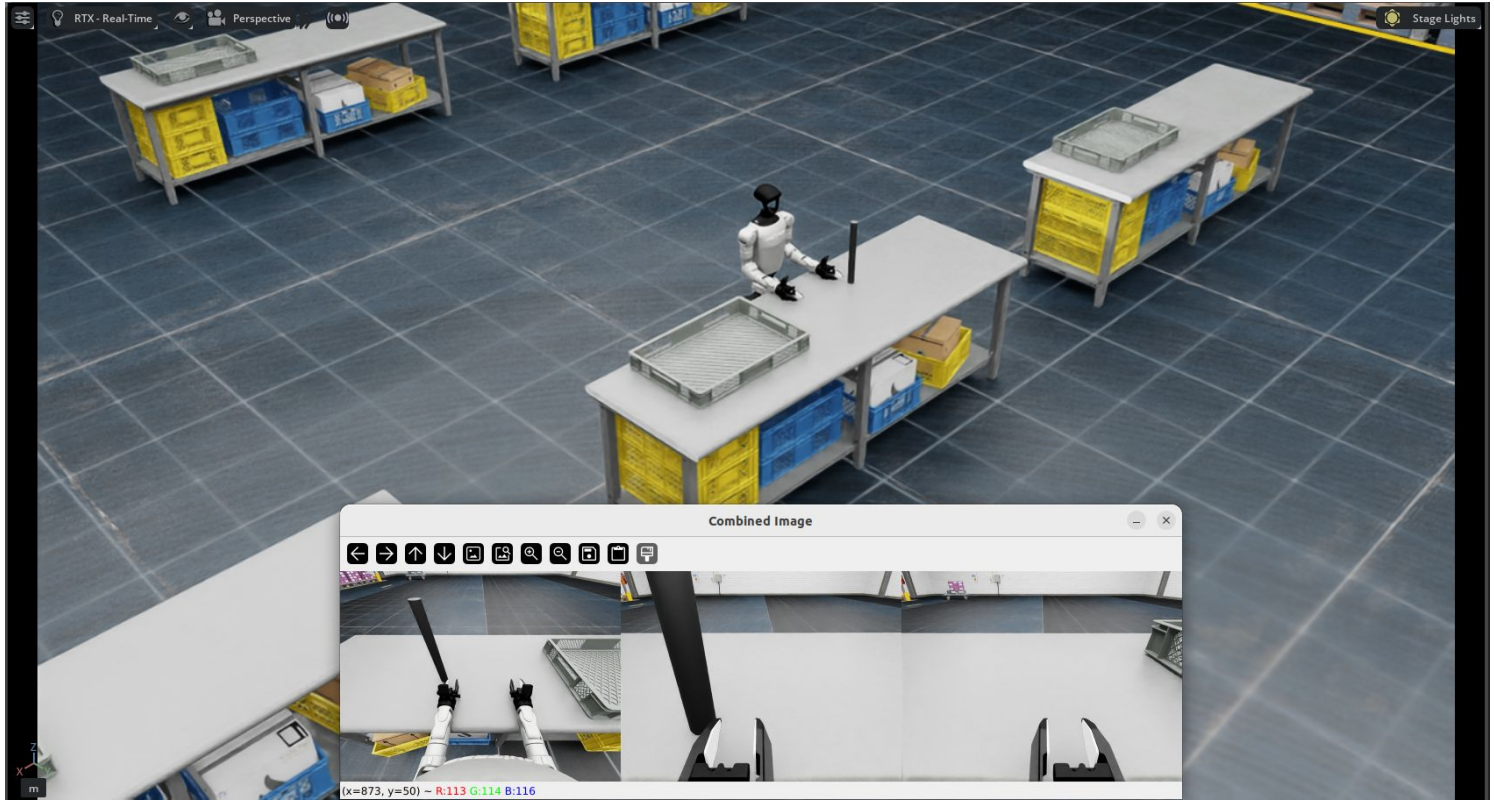
↓

Prostředí

NVIDIA Isaac Sim

- Zajišťuje fyziku
- Vykresluje 3D svět, kamery a světlo

Ukázka prostředí



Zdroj: alza.cz, botland.cz

Podporované platformy



Unitree G1

- Kompaktní humanoid *(130cm)*
- Cenová dostupnost, masová výroba



Unitree H1

- Plnohodnotný humanoidní robot *(180cm)*
- Výzkum a průmyslové nasazení

Zdroj: unitree.com, usrobotstore.com

Podporované úlohy

Stacionární manipulace

- Úchop a manipulace s objekty na stole

Whole-Body ovládání

- Podpora lokomoce i manipulace
- Koordinace rovnováhy

Robotické ruce:



Dex1
Unitree G1

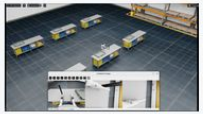
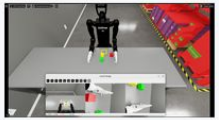
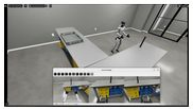


Dex3
Unitree G1



Inspire
Unitree G1+H1

Scénáře

G1-29dof-gripper	G1-29dof-dex3	G1-29dof-inspire	H1-2-inspire
 `Isaac-PickPlace-Cylinder-G129-Dex1-Joint`	 `Isaac-PickPlace-Cylinder-G129-Dex3-Joint`	 `Isaac-PickPlace-Cylinder-G129-Inspire-Joint`	 `Isaac-PickPlace-Cylinder-H12-27dof-Inspire-Joint`
 `Isaac-PickPlace-RedBlock-G129-Dex1-Joint`	 `Isaac-PickPlace-RedBlock-G129-Dex3-Joint`	 `Isaac-PickPlace-RedBlock-G129-Inspire-Joint`	 `Isaac-PickPlace-RedBlock-H12-27dof-Inspire-Joint`
 `Isaac-Stack-RgyBlock-G129-Dex1-Joint`	 `Isaac-Stack-RgyBlock-G129-Dex3-Joint`	 `Isaac-Stack-RgyBlock-G129-Inspire-Joint`	 `Isaac-Stack-RgyBlock-H12-27dof-Inspire-Joint`
 `Isaac-Move-Cylinder-G129-Dex1-Wholebody`	 `Isaac-Move-Cylinder-G129-Dex3-Wholebody`	 `Isaac-Move-Cylinder-G129-Inspire-Wholebody`	

+ Vlastní scénáře

Zdroj: Unitree

xr_teleoperate

- **Manuální a interaktivní řízení** robotů Unitree v reálném čase pomocí **XR** (eXtended Reality) zařízení
- Přímo napojitelné na unitree_sim_isaaclab



03

Instalace simulátoru

Hardware požadavky

Systémové základy

CPU:

Moderní vícejádrový
(Intel i7/i9 12. gen+, AMD Ryzen 7/9 5000+)

Úložiště:

Rychlé NVMe SSD, **min. 50GB**

Paměť

RAM:

Minimálně 32 GB

VRAM:

16 GB VRAM a více
(nutné pro komplexní scény)

Grafická karta

Architektura:
Výhradně **NVIDIA** s podporou **RTX** (CUDA, PhysX, RayTracing)

Řada:
RTX 40xx a vyšší (Tensor jádra 4. gen, nové ovladače)

Otestovaná konfigurace
Běh a scénáře testovány na **NVIDIA RTX 5070 Ti (16 GB VRAM)** – plynulý běh simulace s použitím DLSS.

Požadavky na operační systém

(nevztahuje se na čistý Isaac Sim/Lab)

Windows / WSL2 - NEPODPOROVÁNO

Nativní Windows

- Balíček **Pinocchio (PIN)** nemá oficiální podporu pro Windows
  (potřeba pro inverzní kinematiku humanoidů)
- Thread management simulátoru není na Windows podporován → **Memory Access Violation**

WSL2 (Windows Subsystem for Linux) / Docker

- **Chybějící přímý přístup k GPU:** Chybí plná podpora nízkoúrovňového API **Vulkan**, na kterém Omniverse/Isaac Sim vykresluje. (Podpora pouze pro CUDA)
- **Jediný workaround:** Běh v **Headless módu** (čistě výpočty bez GUI), což ale znemožňuje vizuální teleoperaci a ladění scény.

GNU / Linux

Nativní Linux

- Podporováno
- Doporučována distribuce **Ubuntu 22.04 LTS** a novější
- Možné i deriváty Ubuntu (např. **Linux Mint**)

Docker (Linux)

- Podporováno
- **Komplikace:**
  - Omezený výkon o **20–50 %** (Problémy s GPU passthrough)
  - Cachování shaderů - Při každém novém spuštění kontejneru (řády minut)

Možnosti instalace Isaac Sim

Isaac Sim 5.0.0 / 5.1.0

- Doporučeno pro NVIDIA RTX 50xx
- Ideálně 16GB+ VRAM

Isaac Sim 4.5.0

- Doporučeno pro starší nebo méně výkonné GPU

Poznámka:
Následující kroky v průvodci budou ukazovat postup pro **obě varianty** instalace s **upřednostněním verze 5.1.0**.

Prerekvizity

01 Správný NVIDIA Driver

Ověření pomocí příkazu `nvidia-smi`

Často je nutné přeinstalovat – ovladač nainstalovaný distribucí bývá zastaralý.

02 Verze Pythonu

Vyžadováno: Přesně verze **3.11** (případně 3.11.x)

(Pro Isaac Sim 4.5.0 je nutná verze **3.10**)

03 Python prostředí

Doporučeno použít izolované prostředí:

- **venv** nebo **conda**

04 PyTorch Balíček

Instalace dle oficiálního návodu:

https://pytorch.org/get-started/

Vrstva 1 - NVIDIA Isaac Sim

Příkaz pro instalaci (PyPI)

```
pip install "isaacsim[all,extscache]==5.1.0" --extra-index-url https://pypi.nvidia.com
```

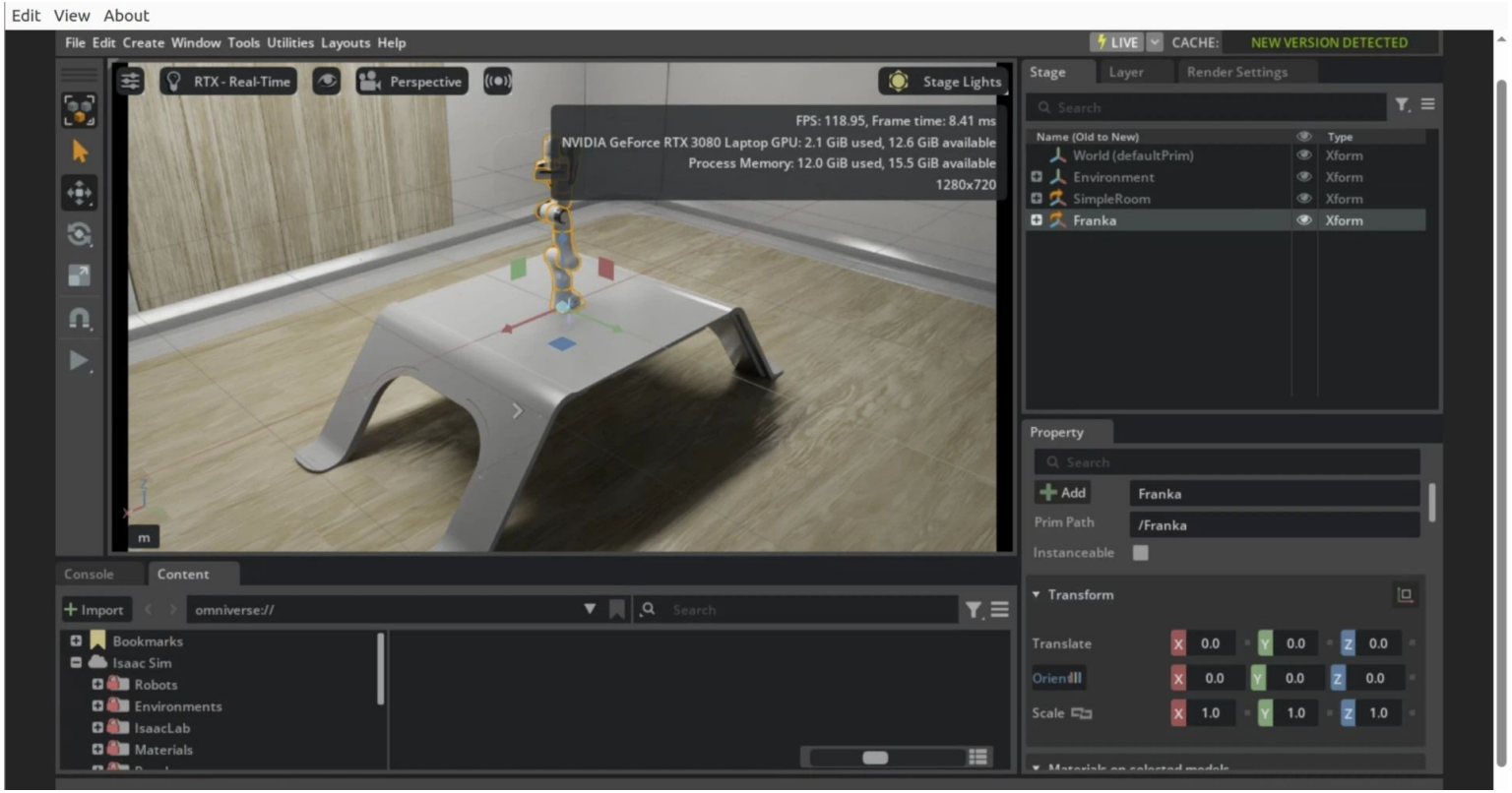
Alternativní verze:

`==5.0.0` nebo `==4.5.0`

Ověření instalace

Úspěšnou instalaci lze ověřit spuštěním příkazu `isaacsim` v daném aktivovaném prostředí.

Ukázka prostředí



Vrstva 2 - NVIDIA Isaac Lab

01 Naklonování repozitáře

Stáhněte si zdrojové kódy IsaacLab z GitHubu:

`isaac-sim/IsaacLab`

03 Instalační script

Spusťte instalaci pomocí připraveného scriptu:

`./isaaclab.sh --install`

02 Přepnutí na správný commit

Nastavte odpovídající verzi kódu pomocí `git checkout` podle vaší verze Isaac Sim:

- 4.5.0: `git checkout 91ad4944f2b7fad29d52c04a5264a082bcaad71d`
- 5.0.0: `git checkout v2.2.0`
- 5.1.0: `git checkout 80094be3245aa5c8376a7464d29cb4412ea518f5`

Ověření instalace

Úspěšnou instalaci lze ověřit spuštěním příkazu `python scripts/tutorials/00_sim/create_empty.py` v daném aktivovaném prostředí.

Vrstva 3a - Unitree SDK

01 Klonování repozitáře

Stáhněte si zdrojové kódy Unitree SDK z GitHubu:

```
git clone
https://github.com/unitreerobotics/unitree_sdk2_python
```

02 Instalace balíčku

Nainstalujte balíček do našeho prostředí:

```
pip install -e .
```

Možné problémy

- **Nekompatibilní balíčky:** Např. pro balíček `numpy` se liší požadavky v SDK a Isaac Sim (lze neřešit).
- **CycloneDDS:** Implementace protokolu DDS.

Oficiální dokumentace uvádí, že Python balíček by měl být schopen nainstalovat základní verzi CycloneDDS automaticky.

– Většinou je však nutná manuální kompilace zdrojových souborů přes `cmake` (**cyclonedds.io**).

Vrstva 3b - unitree_sim_isaaclab

01 Naklonování repozitáře

Stáhněte si zdrojové kódy repozitáře z GitHubu:

```
git clone https://github.com/unitreerobotics/unitree_sim_isaaclab.git
git submodule update --init --depth 1
```

02 Instalace prerekvizit

Nainstalujte potřebné Python balíčky ze souboru requirements:

```
pip install -r requirements.txt
```

Možné problémy

- **Nekompatibilní balíčky:** Např. opět pro balíček `numpy` se líší požadavky (lze opět neřešit).

Vrstva 3c - Teleimager

Balíček pro komunikaci s kamerami (i reálných robotů)

01 Úprava konfiguračního souboru

Upravte hodnoty v souboru `teleimager/cam_config_server.yaml`:

```
image_shape : [480, 640]
type : isaacsim
binocular : false
```

** V nové verzi již nemusí být zapotřebí.*

02 Nastavení Python verze

Upravte soubor `teleimager/pyproject.toml` na současnou Python verzi:

```
requires-python = ">=3.8, <3.12"
```

** V nové verzi již nemusí být zapotřebí.*

03 Instalace teleimageru

Nainstalujte balíček:

```
pip install -e .
```

Oprava kódu

Oprava chybného formátování orientace robota kvůli reprezentaci kvaternionů

Úprava souboru

V souboru `tasks/common_observations/g1_29dof_state.py`

```
# Původní chybný kód:
quat_wxyz = ensure_quat_w_first(quat, assume_w_first = quat_w_first)

# Opravený kód:
quat_wxyz = ensure_quat_w_first(quat, assume_w_first = True)
```

** V nové verzi již nemusí být zapotřebí.*

Stažení assetů

Stažení potřebných 3D modelů, meshů, textur

01 Instalace Git LFS

Nainstalujte rozšíření Git Large File Storage pro stahování velkých binárních souborů:

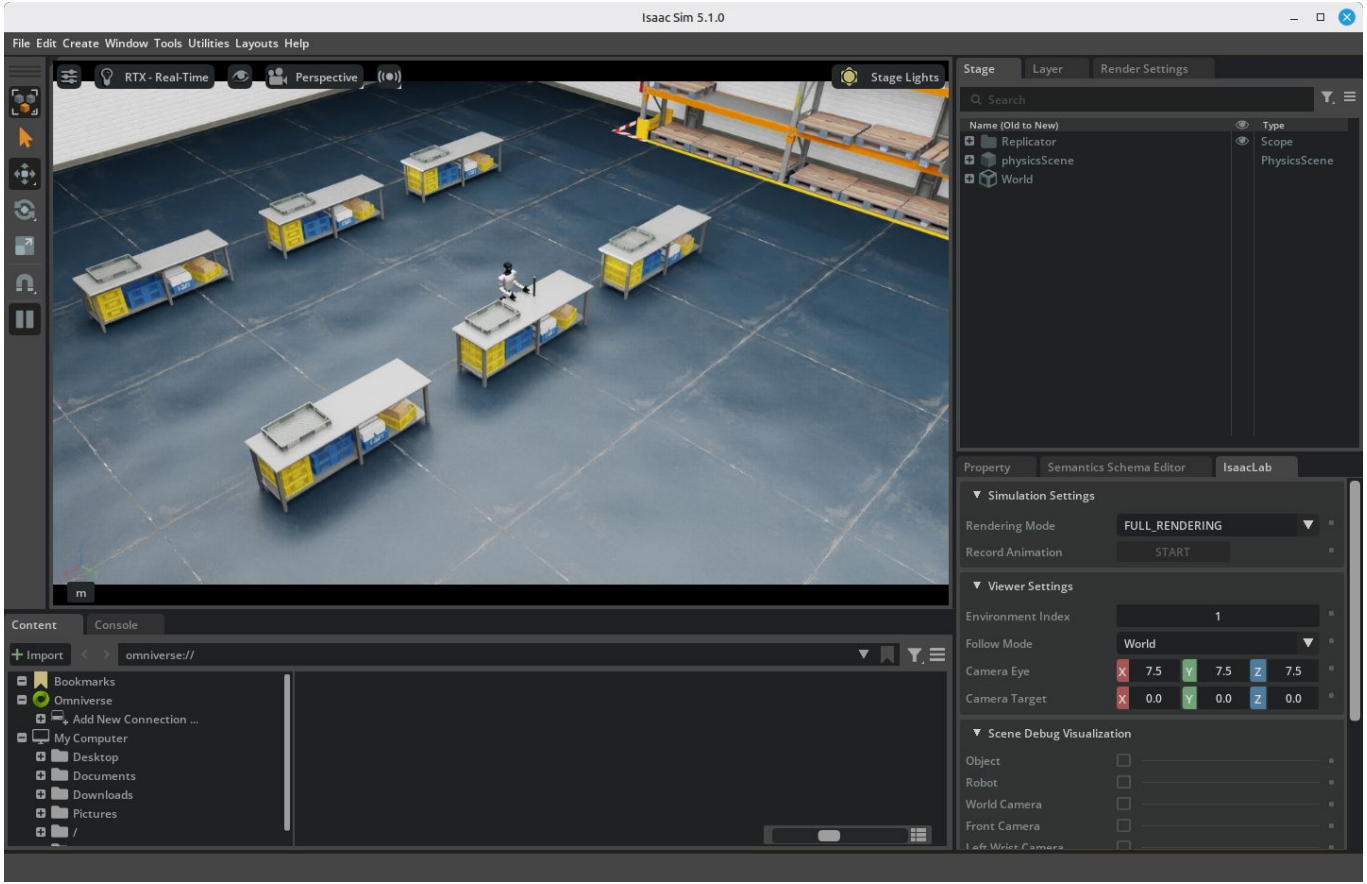
```
sudo apt install git-lfs
```

02 Skript pro assety

Připravený skript pro automatické stažení:

```
./fetch_assets.sh
```

Výsledek


Isaac Sim 5.1.0
File Edit Create Window Tools Utilities Layouts Help
RTX - Real-Time
Perspective
Stage Lights
Stage
Layer
Render Settings
Search
Name (Old to New)
Type
Replicator
Scope
physicsScene
PhysicsScene
World
Property
Semantics Schema Editor
IsaacLab
Simulation Settings
Rendering Mode
FULL_RENDERING
Record Animation
START
Viewer Settings
Environment Index
1
Follow Mode
World
Camera Eye
7.5
7.5
7.5
Camera Target
0.0
0.0
0.0
Scene Debug Visualization
Object
Robot
World Camera
Front Camera
Content
Console
Import
omniverse://
Bookmarks
Omniverse
Add New Connection ...
My Computer
Desktop
Documents
Downloads
Pictures
/
m

Možnosti zjednodušení instalace

01 Automatický install script

Skript pro automatické nastavení prostředí:

`auto_setup_env.sh`

Zůstává ale:

- Instalace CycloneDDS
- Kolize balíčků
- A další...

02 Docker Image

Základní image poskytován přímo v repozitáři.

- Nutno řešit passthrough:

```
xhost +local:docker
sudo docker run --gpus all -it --rm \
--network host \
-e NVIDIA VISIBLE DEVICES=all \
-e NVIDIA DRIVER CAPABILITIES=compute,utility,video,graphics,display \
-e LD LIBRARY PATH=/usr/local/nvidia/lib:/usr/local/nvidia/lib64:$LD_LIBRARY_PATH \
-e DISPLAY=$DISPLAY \
-e VK ICD FILENAMES=/etc/vulkan/icd.d/nvidia_icd.json \
-v /etc/vulkan/icd.d:/etc/vulkan/icd.d:ro \
-v /usr/share/vulkan/icd.d:/usr/share/vulkan/icd.d:ro \
-v /tmp/.X11-unix:/tmp/.X11-unix:rw \
-v
/usr/lib/x86_64-linux-gnu/libnvidia-ngx.so.580.95.05:/usr/lib/x86_64-linux-gnu/libnvidia-ngx.so.580
.95.05:ro \
unitree-sim /bin/bash
```

- Menší výkon o 20–50 %
- Různé chyby (dají se ignorovat)
- Cachování shaderů - Nutno znovu zpracovat při každém znovuvytvoření (řády minut)

04

Použití simulátoru

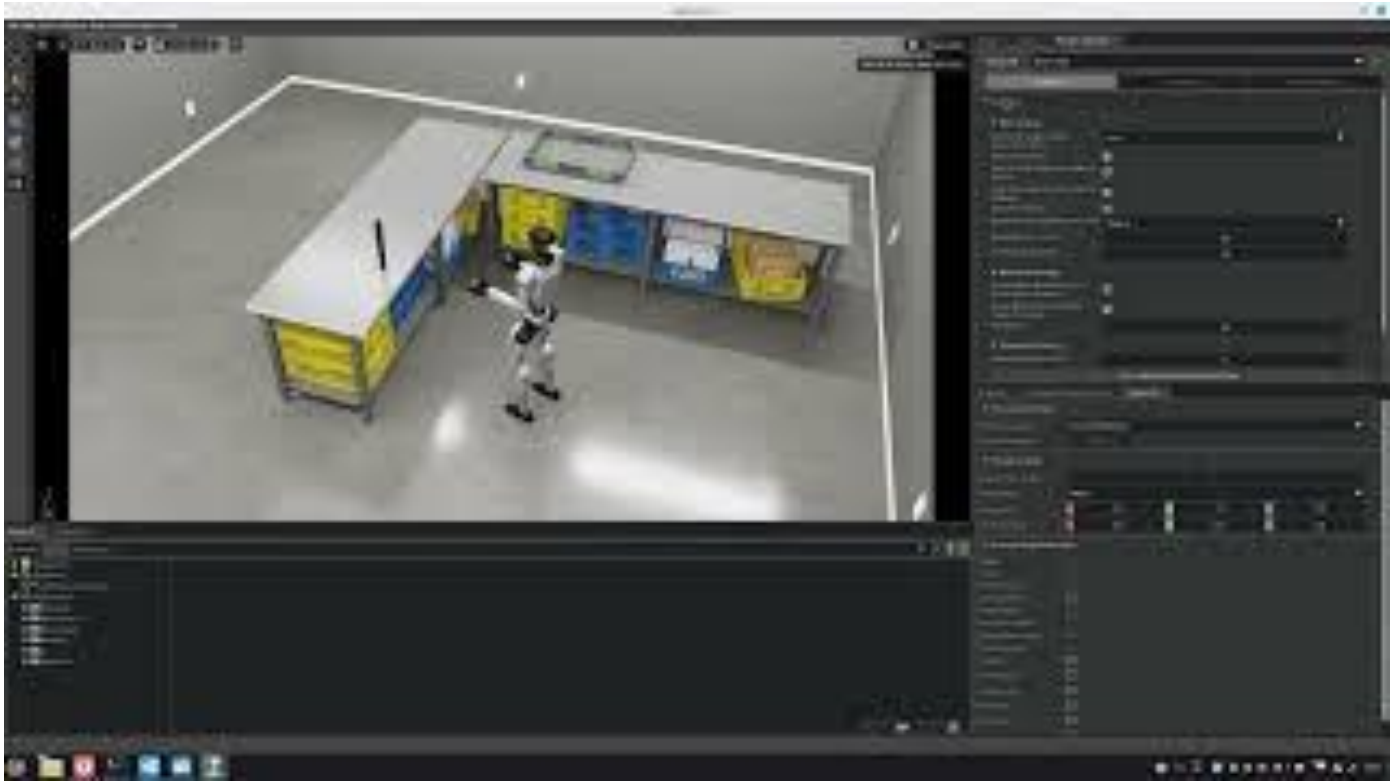
Prostředí



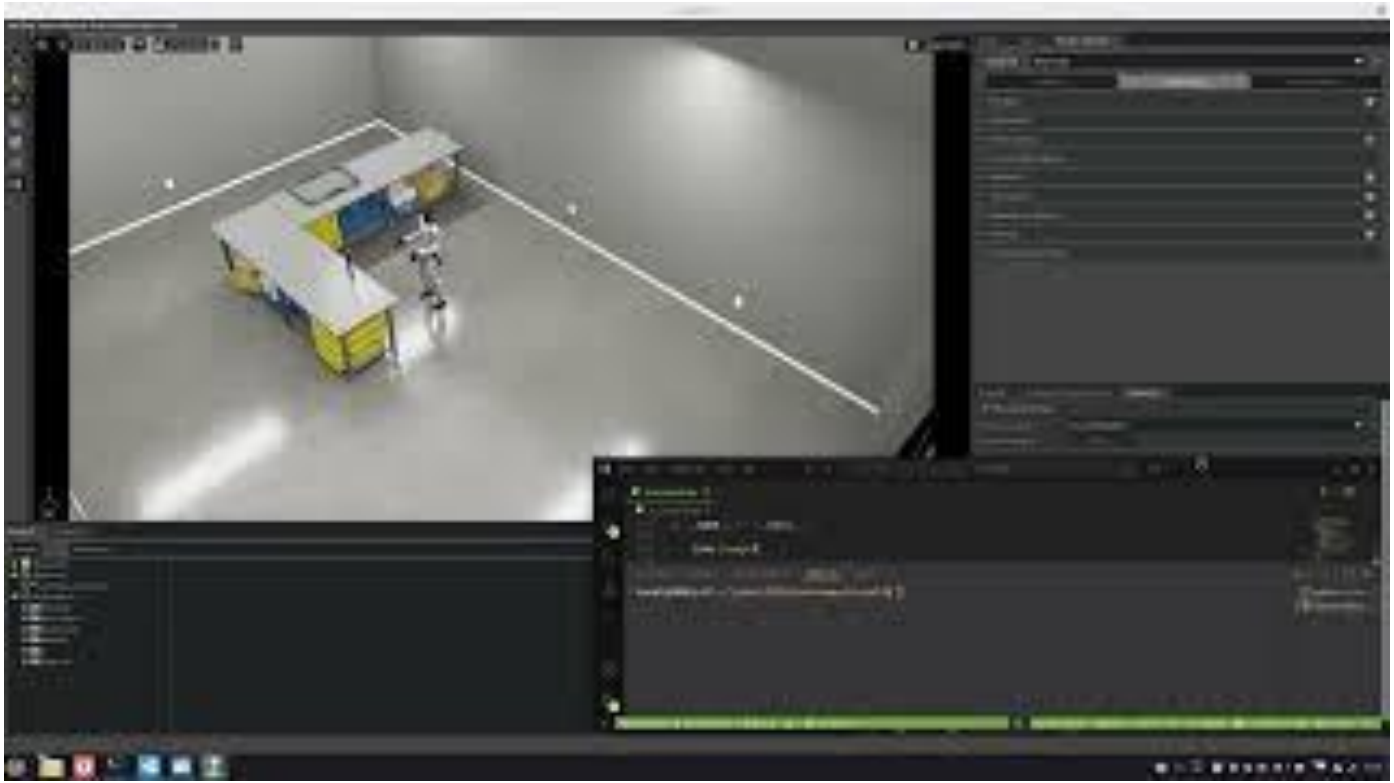
Ovládání robota



Výkon simulátoru



Ovládání robota pomocí scriptu



Ovládání ruky pomocí scriptu



Nevýhody

01 Stabilita

- Velké množství bugů
- Nepřehlednost kódu a dokumentace

02 Absence High Level API

High Level API není přímo podporováno:

- V Unitree SDK se nazývá `SportMode`
  - Má vlastní Client
- V simulátoru existuje pouze ekvivalent: `rt/run_command/cmd`

Důsledek:

- Je nutný zcela jiný přístup přes DDS

Replay & Data Generation

01 Data Replay (Přehrávání trajektorií)

Princip: Zpětné přehrávání reálných záznamů chování robota (typicky datasetů nahraných přes `xr_teleoperate`).

Spuštění: Zajišťují parametry:

- `--replay`
- `--file_path <cesta_k_datasetu>`

02 Data Generation

Princip: Data Augmentation (z jednoho záznamu vytvoří tisíce variant, což zlepšuje generalizaci AI).

Spuštění:

- `--modify_light`: Mění světelné podmínky, stíny a odlesky.
- `--modify_camera`: Mění úhly, FOV nebo přidává šum.

Ukázka využití simulátoru





Děkuji za pozornost

Zdrojové kódy

`https://github.com/KarelRuzicka/unitree_sim_isaaclab_scripts`

